RÉPUBLIQUE DE MADAGASCAR

MINISTÈRE DE L'ENSEIGNEMENT SUPÉRIEUR ET DE LA RECHERCHE SCIENTIFIQUE

UNIVERSITÉ DE FIANARANTSOA

ÉCOLE DE MANAGEMENT ET D'INNOVATION TECHNOLOGIQUE (EMIT)

Département : Développement d'Application Internet et Intranet (DASI)

[Emplacements réservés aux logos : Université de Fianarantsoa — EMIT — Établissement d'accueil]

RAPPORT DE STAGE

En vue de l'obtention du diplôme de Licence en informatique, parcours Développement d'Application Internet et Intranet (DASI)

TITRE DU MÉMOIRE

Conception et développement d'une application web de digitalisation de la gestion des plannings du personnel d'un établissement hôtelier

Cas de l'établissement Zomatel (hôtellerie – restauration)

Présenté et soutenu par :

« Nom et prénoms de l'étudiant : À COMPLÉTER »

Période de stage : À COMPLÉTER — Maître de stage (entreprise) : À COMPLÉTER

Établissement d'accueil : ZOMATEL — adresse : À COMPLÉTER

Encadrant pédagogique : À COMPLÉTER

Membres du jury : À COMPLÉTER (président) — À COMPLÉTER (rapporteur)

Année universitaire : À COMPLÉTER

PAGE DE GARDE

(

Version noir et blanc, à imprimer recto uniquement — en-tête de page : logos ; contenu identique à la page de couverture.

N.B. : conformément au canevas, la page de garde reprend fidèlement la couverture, en noir et blanc.

Signe distinctif / fac-similé de signature facultatif : À COMPLÉTER

CURRICULUM VITAE

Photo récente (tenue correcte) : À COMPLÉTER

Nom et prénoms : À COMPLÉTER

Né(e) le : À COMPLÉTER — à À COMPLÉTER

Adresse : À COMPLÉTER — Téléphone : À COMPLÉTER — Courriel : À COMPLÉTER

Diplômes et formations :

1. À COMPLÉTER — année universitaire À COMPLÉTER
2. À COMPLÉTER
3. À COMPLÉTER

Expériences et stages :

4. Stage de À COMPLÉTER — À COMPLÉTER
5. Stage 3e année (Licence DASI) : Établissement ZOMATEL — digitalisation de la gestion des plannings du personnel

Compétences :

- Développement web full stack : JavaScript (Next.js, React), Node.js/Express ;
- Bases de données relationnelles : MariaDB (modélisation, requêtes SQL, triggers et vues) ;
- Analyse et conception : UML, méthode de conception orientée objets ;
- Gestion de versions : Git.

REMERCIEMENTS

Avant toute chose, je tiens à exprimer ma profonde gratitude envers tous ceux qui, de près ou de loin, ont contribué au bon déroulement de ce stage et à l'élaboration du présent rapport.

Je remercie sincèrement mon maître de stage au sein de l'établissement ZOMATEL, À COMPLÉTER, pour son accueil, pour la confiance qu'il m'a accordée et pour les multiples explications qui ont permis de cerner au plus près les réalités opérationnelles de la gestion des plannings du personnel de son établissement.

Je remercie également mon encadrant pédagogique, À COMPLÉTER, pour la rigueur de ses consignes, pour la pertinence de ses remarques et pour la disponibilité dont il a fait preuve tout au long de la réalisation de ce travail.

Mes remerciements s'adressent aussi à l'ensemble du corps enseignant de l'EMIT, département DASI, pour la qualité de la formation reçue, ainsi qu'à l'administration de l'école pour son organisation et son suivi.

Enfin, je remercie mes camarades de promotion, le personnel des services de l'établissement ZOMATEL que j'ai eu à fréquenter pendant la phase d'analyse (cuisine, salle, réception, administration), ainsi que ma famille, pour leur soutien constant.

(Personnaliser librement les personnes citées : À COMPLÉTER.)

RÉSUMÉ

Dans l'hôtellerie-restauration, l'élaboration des plannings hebdomadaires du personnel reste, dans de nombreux établissements, un exercice essentiellement manuel : tableurs indépendants par service, calculs de heures faits de tête, transmissions par messagerie instantanée, et absence de circuit de validation formalisé. Ce fonctionnement engendre des erreurs de totaux, des conflits d'horaires non détectés, une couverture de service parfois insuffisante et une traçabilité quasi nulle des modifications.

Le présent rapport décrit la conception et le développement d'une application web destinée à digitaliser la gestion des plannings du personnel d'un établissement hôtelier, l'établissement ZOMATEL. L'application, structurée en trois tiers (interface Next.js/React, API REST Node.js/Express, base de données MariaDB), permet à chaque chef de service d'élaborer le planning hebdomadaire de son service dans une grille directement comparable à un tableur, avec calcul automatique des heures travaillées par employé (vues SQL) et comparaison à une référence hebdomadaire individualisée (44 h ou 48 h). Un circuit de validation formalisé relie ensuite le chef de service à la RH : soumission, contrôle, validation ou rejet motivé, avec notifications automatiques et historique horodaté de toutes les actions.

L'application couvre trois formats de saisie configurables par service (périodes fixes matin/après-midi/soir, horaire libre, shift par poste), la gestion des congés et récupérations, le contrôle de couverture par créneau, l'export des emplois du temps en Excel et une messagerie interne. Les règles de gestion sont appliquées au plus près des données, par des triggers MariaDB, en complément des contrôles réalisés côté serveur et côté client.

MOTS-CLÉS : planning du personnel ; hôtellerie ; digitalisation ; application web ; Next.js ; Node.js ; MariaDB ; validation RH ; heures de travail.

ABSTRACT

In hotel and restaurant establishments, the weekly scheduling of staff still relies, in many places, on a mostly manual exercise: separate spreadsheets per department, mental calculation of worked hours, ad-hoc transmission through messaging apps, and no formal validation circuit. This leads to inaccurate totals, undetected schedule conflicts, insufficient service coverage and a nearly nonexistent audit trail of changes.

This report describes the design and development of a web application to digitize the staff scheduling of a hotel establishment, ZOMATEL. Built as a three-tier architecture (a Next.js/React interface, a Node.js/Express REST API and a MariaDB database), the application lets each department manager build the weekly schedule of his department in a spreadsheet-like grid, with automatic computation of worked hours per employee (SQL views) and a comparison against an individual weekly reference (44 h or 48 h). A formal validation workflow then connects the department manager with the HR office: submission, review, approval or rejection with justification, automatic notifications and a time-stamped audit log of every action.

The application supports three input formats configurable per department (fixed periods morning/afternoon/evening, free-hour grid, shift per station), the management of leave and compensations, coverage control per time slot, Excel exports of schedules and an internal messaging module. Business rules are enforced close to the data, through MariaDB triggers, in addition to server-side and client-side checks.

KEYWORDS: staff scheduling; hotel industry; digitization; web application; Next.js; Node.js; MariaDB; HR validation; working hours.

LISTE DES FIGURES

Figure 1.1 — Chronogramme de réalisation du projet (diagramme de Gantt).

Figure 3.1 — Architecture trois tiers de l'application.

Figure 4.1 — Diagramme de flux du processus actuel de gestion des plannings.

Figure 4.2 — Diagramme de cas d'utilisation de l'application.

Figure 5.1 — Diagramme de séquence système : soumission et validation d'un planning.

Figure 5.2 — Diagramme de séquence système : authentification.

Figure 5.3 — Modèle de domaine.

Figure 6.1 — Diagramme de paquetages de l'application.

Figure 6.2 — Diagramme de déploiement.

Figure 6.3 — Modèle conceptuel des données (MCD).

Figure 6.4 — Diagramme de classes backend.

LISTE DES TABLEAUX

Tableau 3.1 — Moyens humains, matériels et logiciels nécessaires au projet.

Tableau 3.2 — Résultat attendu (livrables).

Tableau 4.1 — Critique de l'existant.

Tableau 5.1 — Extrait du dictionnaire de données.

Tableau 5.2 — Règles de gestion.

Tableau 5.3 — Description textuelle du cas d'utilisation « Soumettre un planning à la RH ».

Tableau 5.4 — Description textuelle du cas d'utilisation « Valider ou rejeter un planning soumis ».

Tableau 6.1 — Synthèse de la base de données gestion_plannings_hotel.

Tableau 6.2 — Principales règles métier appliquées par triggers MariaDB.

Tableau 7.1 — Outils et technologies de réalisation.

Tableau 9.1 — Comparaison de l'application avec des solutions existantes.

LISTE DES SIGLES ET ABRÉVIATIONS

API : Application Programming Interface (interface de programmation applicative)

BROUILLON : Premier état d'un planning, librement modifiable par le chef de service

CRF : Sigle non utilisé dans le corps du rapport (conservé comme entrée vide)

DASI : Développement d'Application Internet et Intranet (parcours de licence)

EMIT : École de Management et d'Innovation Technologique

ENUM : Type de colonne SQL n'acceptant qu'une liste finie de valeurs

HR / RH : Human Resources / Ressources Humaines

HTTP : HyperText Transfer Protocol (protocole de transfert du web)

JWT : JSON Web Token (jeton d'authentification)

MCD : Modèle Conceptuel des Données

MVC : Modèle-Vue-Contrôleur (motif d'architecture logicielle)

REST : REpresentational State Transfer (style d'architecture d'API web)

RH : Ressources Humaines

SDK : Software Development Kit

SOUMIS : État d'un planning transmis à la RH et en attente de décision

SQL : Structured Query Language (langage de requête des bases de données)

UML : Unified Modeling Language (langage de modélisation unifié)

VALIDE / REJETE : États finals d'un planning après décision de la RH

(La liste complète des états d'un planning est donnée dans la règle de gestion n°5 : À COMPLÉTER à ajuster si nécessaire.)

GLOSSAIRE

Affectation : Statut d'un employé pour un jour donné d'un planning (TRAVAIL, REPOS, ABSENCE).

Besoins de couverture : Effectifs requis par jour et par créneau pour un service.

Couverture : Comparaison entre l'effectif requis et l'effectif réel previsto d'un créneau de service.

Créneau de travail : Période horizontale d'une journée (matin, après-midi, soir) avec heure d'entrée et heure de sortie.

Emploi du temps : Planning hebdomadaire validé, transformé en support de consultation et d'export Excel.

Grille du planning : Vue d'édition directe employé × jour du lundi au dimanche, façon tableur.

Message : Contenu textuel ou fichier échangé dans une conversation de la messagerie interne.

Notification : Message généré automatiquement par le système pour informer un utilisateur (décision, soumission).

Notification temps réel : Notification poussée en direct au client connecté via socket.io.

Permission : Droit d'accès associée à un rôle (côté serveur, chaque route vérifie sa permission requis).

Récupération : Heure de repos compensé enregistrée par le chef de service ou la RH.

Référence hebdomadaire : Volume d'heures de référence d'un employé (44 h ou 48 h) utilisé pour comparaison.

Service : Département de l'établissement (Cuisine Snack, Salle Snack, Patisserie & Pizza, ...).

Trigger : Programme SQL exécuté automatiquement au moment d'un insert ou d'un update en base.

Vue SQL : Requête enregistrée en base, calculant par exemple la durée totale d'un employé.

TABLE DES MATIÈRES

TOC \o "1-3" \h \z \uTable des matières : appuyez sur Ctrl+A puis F9 (ou clic droit > Mettre à jour les champs) pour l'actualiser dans Word.

INTRODUCTION GÉNÉRALE

Parmi les établissements du secteur de l'hôtellerie-restauration, la qualité de l'accueil offert à la clientèle dépend en grande partie d'une donnée souvent invisible : la bonne répartition du personnel sur les différentes plages de service. Cuisine, salle, réception, étage ou encore détente : chaque service doit être doté chaque jour et pour chaque créneau horaire des effectifs nécessaires, tout en respectant les contraintes individuelles du personnel —repos hebdomadaire, congés, stagiaires sans matricule, références de 44 ou 48 heures par semaine.— Dès lors, la construction du planning hebdomadaire est un exercice aussi délicat qu'essentiel.

Dans la pratique, cet exercice reste dans de nombreux établissements purement manuel. Un tableur indépendant est rédigé par service, les durées de chaque créneau et les totaux hebdomadaires sont calculés de tête ou au tableur, et les fichiers circulent de main en main ou par messagerie instantanée. Quelques difficultés apparaissent alors régulièrement : erreurs de calcul des heures, conflits d'horaires non détectés, absence de circuit de validation formel, impossibilité de savoir qui a modifié quoi et quand, et quantité d'impressions papier qui doit être redondée dès qu'une correction intervient.

En se fondant sur ces constats, ce travail pose une hypothèse simple : la digitalisation de la gestion des plannings du personnel, appuyée sur une application web adaptée aux contraintes réelles de l'établissement, permet d'automatiser les calculs, de sécuriser un circuit de validation formel entre les chefs de service et la RH, et de rendre les plannings consultables, traçables et exportables. L'objectif de ce mémoire est d'analyser, concevoir et développer une telle application pour l'établissement ZOMATEL, selon les règles de gestion observées sur le terrain et formalisées dans le cahier des charges.

Le présent rapport est organisé en trois grandes parties. La première présente le cadre général du stage : l'école, l'établissement d'accueil et le projet lui-même, avec ses objectifs et les moyens mis en œuvre. La deuxième expose l'analyse et la conception : étude de l'existant, choix techniques, règles de gestion, diagrammes UML et modèle de données. La troisième décrit la réalisation : architecture mise en place, configuration, fonctionnalités développées, présentation de l'application, puis une évaluation critique assortie de limites et de perspectives.

PARTIE 1 : PRÉSENTATION GÉNÉRALE

CHAPITRE 1 : PRÉSENTATION DE L'ÉCOLE

1.1. Historique et rattachement

L'École de Management et d'Innovation Technologique (EMIT) est un établissement de l'Université de Fianarantsoa. À COMPLÉTER ; les éléments suivants sont à ajuster d'après les données officielles de l'école.

L'EMIT forme des cadres dans les domaines du management de la technologie et de l'informatique, avec plusieurs parcours de licence et de master. Le présent rapport a été réalisé dans le cadre du parcours de licence en informatique, spécialité Développement d'Application Internet et Intranet (DASI), qui forme les étudiants à la conception et la réalisation d'applications web couvrant l'ensemble des couches (interface utilisateur, logique métier, base de données).

1.2. La formation DASI et le stage de troisième année

La formation DASI cèle plusieurs axes utiles à ce projet :

- la programmation orientée objet et les langages du web (JavaScript, PHP, Java) ;
- les bases de données relationnelles et la production de requêtes SQL ;
- l'analyse et la conception informatiques, avec une grande place donnée à la modélisation UML ;
- les méthodes et techniques de gestion de projet informatique.

Le stage de troisième année constitue la mise en situation de ces acquis : chaque étudiant réalise, au sein d'une structure d'accueil, un projet informatique complet de la collecte des besoins jusqu'au compte rendu détaillé présenté dans un rapport comme celui-ci.

CHAPITRE 2 : PRÉSENTATION DE L'ENTREPRISE D'ACCUEIL

2.1. Présentation générale

ZOMATEL est un établissement d'hôtellerie-restauration comportant plusieurs services opérationnels parmi lesquels, du point de vue de la planification du personnel :

- Cuisine Snack ;
- Salle Snack ;
- Patisserie & Pizza.
- ainsi que d'autres services internes (réception, étage, détente, administration…).

Chaque service dispose de son équipe, de ses horaires de service et de ses plages de répartition. Le personnel est principalement de deux statuts : titulaires, possédant un matricule, et stagiaires, simplement identifiés par leur nom. La création des plannings hebdomadaires se fait, dans les pratiques observées avant le projet, au moyen de fichiers Excel indépendants par service.

2.2. Organisation de la gestion du personnel

Adresse complète, historique, effectif global et détails administratifs : À COMPLÉTER. À partir des observations effectuées pendant le stage, la gestion du personnel fonctionne autour de deux acteurs clés : le chef de chaque service, qui connaît les attributions réelles de son équipe, et le bureau des ressources humaines (RH), qui supervise l'ensemble de l'établissement, contrôle les heures travaillées et prépare les éléments de la paie. Ceci correspond exactement à la structure de rôles retenue pour l'application (cf. Chapitre 5).

2.3. Quelques repères

Tableau 2.1 — Repères sur l'établissement d'accueil

Élément	**Information**
Dénomination	ZOMATEL (établissement d'hôtellerie-restauration)
Activité	Hébergement, restauration (snack, salle, patisserie et pizza détente…)
Services planifiés par l'application	Cuisine Snack ; Salle Snack ; Patisserie & Pizza (extensible sans modification du modèle de données)
Effectif et coordonnées	À COMPLÉTER
Contact	À COMPLÉTER

CHAPITRE 3 : PRÉSENTATION DU PROJET

3.1. Formulation du projet

Le projet consiste à concevoir et développer une application web de digitalisation de la gestion des plannings du personnel de l'établissement ZOMATEL. Concrètement, il s'agit de remplacer les fichiers Excel indépendants par une application centralisée permettant :

- la création, par service, de plannings hebdomadaires avec saisie des statuts de chaque jour (travail, repos, absence) et jusqu'à trois créneaux horaires par journée (matin, après-midi, soir) ;
- le calcul automatique des heures travaillées, journalier et hebdomadaire, sans aucune saisie manuelle de totaux ;
- un circuit de validation formel BROUILLON → SOUMIS → VALIDÉ / REJETÉ entre le chef de service et la RH ;
- la traçabilité horodatée de toute création ou modification significative d'un planning ;
- le contrôle automatique de la couverture de service, par comparaison entre l'effectif requis et l'effectif planifié.

3.2. Justification de l'importance du sujet

La gestion des plannings influence directement la qualité de service —un créneau sous-effectif complique le travail de toute l'équipe—, la paie —les heures travaillées doivent être exactes—, et le climat social, puisque les repos et récupérations doivent être appliqués de manière régulière. Aujourd'hui, tout repose sur la rigueur individuelle de chaque responsable, ce qui rend le processus fragile. Le rendu fiable, traçable et automatisable de ces plannings constitue donc un besoin réel, récurrent et quotidien pour l'établissement.

3.3. Objectifs et besoins utilisateurs

3.3.1. Objectif général

Digitaliser de bout en bout la gestion des plannings hebdomadaires du personnel de l'établissement ZOMATEL, en automatisant les calculs d'heures et en garantissant un circuit de validation fiable entre chefs de service et RH.

3.3.2. Objectifs spécifiques

6. Remplacer les plannings papier/Excel par un outil numérique centralisé ;
7. Automatiser le calcul des heures travaillées (aucune saisie manuelle de totaux) ;
8. Mettre en place un circuit de validation formel entre le chef de service et la RH ;
9. Assurer une traçabilité des modifications apportées à un planning ;
10. Contrôler la couverture de service par créneau horaire.

3.3.3. Besoins utilisateurs

Les besoins utilisateurs sont recueillis auprès des deux acteurs concernés :

- Côté chef de service : élaborer en peu de temps le planning de la semaine, en mode « tableur », voir immédiatement les totaux d'heures et les alertes, soumettre le planning à la RH, gérer les fiches de ses employés, saisir les récupérations.
- Côté RH : consulter tous les plannings, contrôler les heures, valider ou rejeter avec un commentaire, créer des comptes utilisateurs, consulter l'historique, exporter les emplois du temps en Excel.

3.4. Moyens nécessaires à la réalisation du projet

Tableau 3.1 — Moyens humains, matériels et logiciels nécessaires au projet

Catégorie	**Moyens**
Humains	Étudiant stagiaire (analyse, conception, développement) ; maître de stage ; personnel concerné (chefs de service, RH) pour les phases d'interview et de validation
Matériels	Ordinateur portable de développement ; serveur local de développement (Laragon) ; poste de test du navigateur web
Logiciels	Node.js (runtime) ; Next.js 14 ; Express ; MariaDB ; Git pour la gestion des versions ; PlantUML et Visual Studio Code pour l'outillage
Jeu de données	Import des plannings de référence du fichier « EMPLOI DU TEMPS ZOMATEL WEEK 37_2026.xlsx » et script d'import en base

3.5. Chronogramme de réalisation du projet (diagramme de Gantt)

Le stage s'est déroulé sur plusieurs semaines, selon l'enchainement analyse → conception → développement → tests. Le chronogramme ci-dessous récapitule les principales phases ; les dates exactes, ajustables, sont à confirmer avec le carnet de bord du stagiaire.

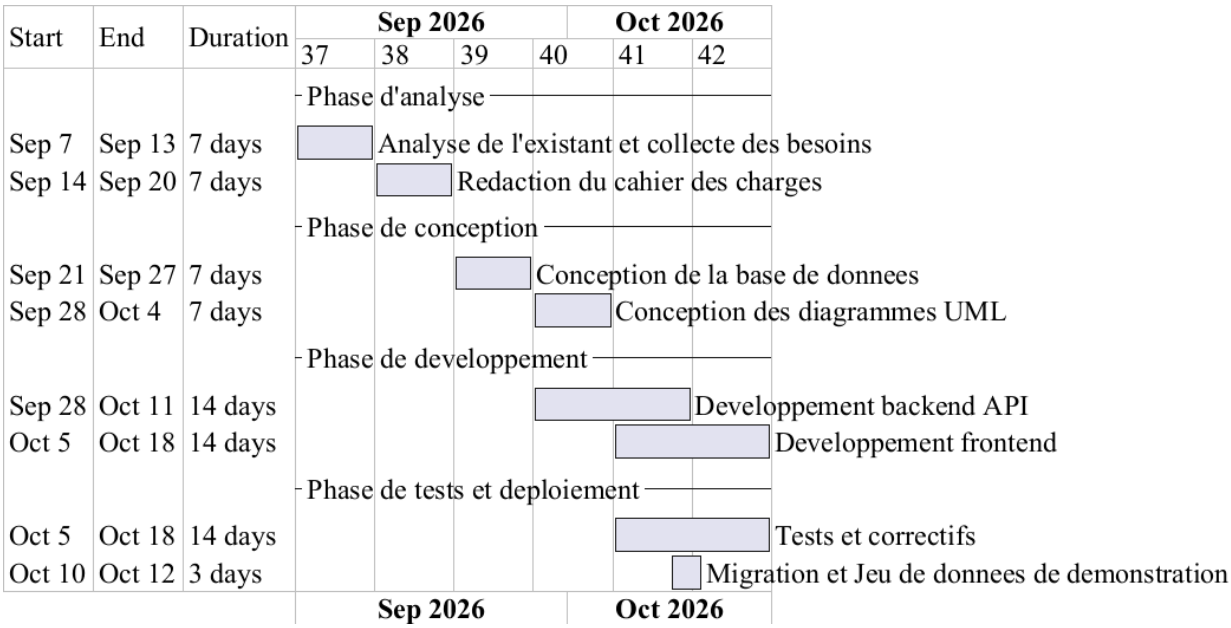


Figure 1.1 — Chronogramme de réalisation du projet (diagramme de Gantt)

(Dates exactes du stage : À COMPLÉTER.)

3.6. Résultat attendu

Tableau 3.2 — Résultat attendu (livrables)

N°	Livrable
1	Script de création de la base de données MariaDB (schéma + vues + triggers + jeu de données de démonstration)
2	API backend Node.js/Express documentée, avec authentification JWT et permissions par rôle
3	Application frontend Next.js (authentification, gestion des plannings, empledur, validation RH, gestion des employés, messagerie interne, notifications)
4	Jeu de données de démonstration et scripts de migration
5	Le présent rapport de stage

PARTIE 2 : ANALYSE ET CONCEPTION

CHAPITRE 4 : ANALYSE PRÉALABLE

4.1. Analyse de l'existant

4.1.1. Organisation actuelle (diagramme de flux)

L'observation du processus actuel, réalisée à partir des fichiers de l'établissement (fichier « EMPLOI DU TEMPS ZOMATEL WEEK 37_2026.xlsx », feuilles « CUISINE SNACK », « SALLE SNACK », « PATISS-PIZZA », etc.), et de la pratique de terrain a permis d'identifier le flux suivant :

1. Chaque responsable de service établit, chaque semaine, un emploi du temps de son équipe dans une feuille du fichier Excel de la semaine ;

2. Pour chaque employé, et pour chaque créneau (matin, après-midi, soir), sont indiqués les heures d'entrée et de sortie, avec le calcul manuel de la durée ;

3. Le fichier circule (WhatsApp, impression, dépôt papier) pour consultation et renseignement de la couverture de service ;

4. La RH consolide manuellement les heures travaillées hebdomadaires par employé pour la paie.

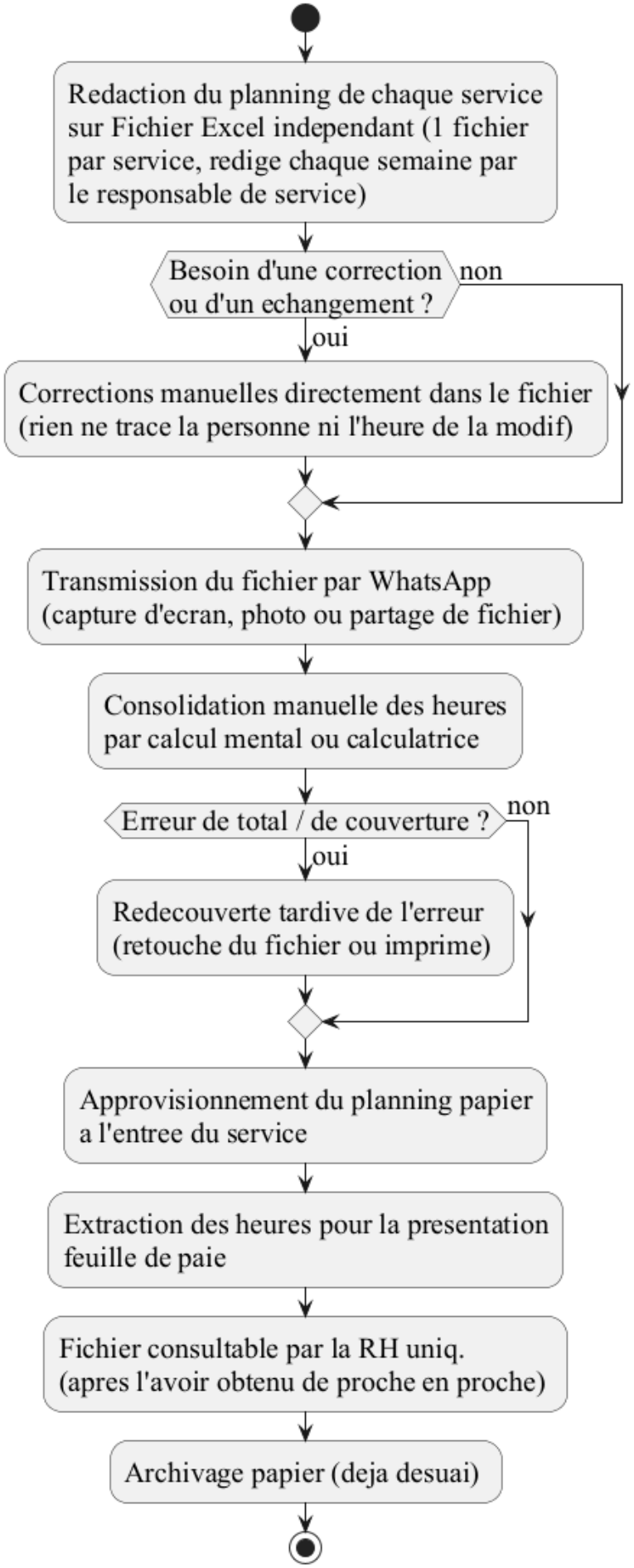

Redaction du planning de chaque service sur Fichier Excel independant (1 fichier par service, redige chaque semaine par le responsable de service)
Besoin d'une correction ou d'un echangement ?
non
oui
Corrections manuelles directement dans le fichier (rien ne trace la personne ni l'heure de la modif)
Transmission du fichier par WhatsApp (capture d'ecran, photo ou partage de fichier)
Consolidation manuelle des heures par calcul mental ou calculatrice
Erreur de total / de couverture ?
non
oui
Redecouverte tardive de l'erreur (retouche du fichier ou imprime)
Approvisionnement du planning papier a l'entree du service
Extraction des heures pour la presentation feuille de paie
Fichier consultable par la RH uniq. (apres l'avoir obtenu de proche en proche)
Archivage papier (deja desuai)

Figure 4.1 — Diagramme de flux du processus actuel de gestion des plannings

4.1.2. Critique de l'existant

Tableau 4.1 — Critique de l'existant

Aspect	Fonctionnalité actuelle	Limites ensuite
Support	Fichier Excel par service, feuilles hebdomadaires	Autant de fichiers que de services x semaines ; pas de vue d'ensemble
Calcul des heures	Formules de tableur, parfois calcul mental	Erreurs de copier-coller ou d'arrondi ; rien en cas de cellule vide ; coût de la correction manuelle
Repos / absence	Mention manuelle dans la cellule	Pas de suivi systématique des repos hebdomadaires et des congés
Circuit de validation	Échange de fichiers ou d'impressions	Pas de circuit formalisé ; pas de statut clair (brouillon / soumis / valide) ; pas d'avis motivé
Traçabilité	Aucune	Impossible de savoir qui a modifié quoi et quand
Couverture de service	Vérification visuelle	Aucun réglage formalisé de l'effectif requis par créneau
Export à la paie	Consolidation manuelle à partir des feuilles Excel	Risque de divergence par rapport aux plannings réellement modifiés

4.1.3. Moyens existants pour la réalisation du projet

- Moyens humains : chefs de service connaisseurs des besoins réels, bureau RH disponible, équipe informatique légère interne (partagée avec d'autres projets) ;
- Moyens matériels : un parc informatique de gestion existant, des ordinateurs portables personnels, le réseau local de l'établissement ;
- Moyens logiciels : suite bureautique déjà utilisée (Excel), serveur local de développement Laragon, outil de gestion de versions Git.

4.2. Choix de la solution

Face à ces limites, deux grandes solutions ont été examinées : (1) rester sur des fichiers Excel partagés, en les homogénéisant par des modèles de feuilles, et (2) développer une application web dédiée. La première solution ne règle ni le calcul fiable des heures, ni le circuit de

validation, ni la traçabilité. La seconde demande un investissement de développement plus important, mais adresse précisément tous les reproches formulés ci-dessus, et permet de faire évoluer les règles de gestion (ajouter un service, un créneau, un indicateur) sans avoir à refaire un modèle de fichier. La solution retenue est donc la deuxième.

4.3. Choix de la méthode de conception / gestion de projet

Le projet a été conduit de manière incrémentale : l'analyse de l'existant et la collecte des besoins ont fourni une base de règles de gestion formalisées dans un cahier des charges ; la conception a été menée selon la méthode UML (Unified Modeling Language), avec une approche orientée objets cohérente avec le développement en JavaScript ; les développements se sont faits par itérations courtes, chaque lot de fonctionnalités étant testé et corrige avant le suivant. Le canevas impose en effet la production d'une série de diagrammes UML (cas d'utilisation, séquence, classes, activité, paquetage, déploiement), ce qui a guidé la méthode de conception.

4.4. Choix des outils de conception

- PlantUML pour les diagrammes (canevas conforme) et Visual Studio Code / IntelliJ comme environnement d'édition ;
- Draw.io et StarUML comme alternatives testées, écartées au profit de solutions en textual-généré (plus faciles à versionner avec Git).

4.5. Choix du langage de programmation

Le langage retenu est JavaScript, exécuté côté serveur par Node.js et côté client par React. Ce choix canalise un certain nombre d'avantages concrets :

- un langage unique pour le frontend et le backend, ce qui simplifie les échanges autour d'une API REST en JSON ;
- un grand toucher de libraries modernes pour la construction rapide d'interfaces riches ;
- un runtime Node.js adapté au développement d'une API REST évolutive et l'accès natif aux bases de données SQL par des libraries dédiées (mysql2).

PHP aurait pu être envisagé (Laragon est un serveur Apache/PHP/MySQL), mais il ne permet pas de partager un design d'application aussi bien arrondi entre le client et le serveur, et l'écosystème actuel autour de React/Next.js est plus favorable au développement d'interfaces de saisie en grille de type tableur.

4.6. Choix du Framework

- Next.js 14 (React) côté frontend : framework officiel recommandé par React pour la construction d'applications web, avec routeur applicatif (App Router), rendu côté serveur pratique, et organisation de code en composants réutilisables ;
- Express côté backend : framework minimaliste et de référence en Node.js pour servir une API REST, avec un écosystème de middlewares adapté (authentification JWT, autorisation par permission, chargement de fichiers, journalisation Morgan).

4.7. Choix de l'environnement de développement

Le développement s'est fait sous Windows avec l'environnement local Laragon (Apache, MariaDB, Node.js), Visual Studio Code comme éditeur, Git pour la gestion de versions, et Google Chrome pour les tests d'affichage. Le frontend est servi sur le port 3000 (next dev), l'API Express sur le port 4000.

4.8. Choix du système de gestion de base de données

Le système retenu est MariaDB, la variante communautaire de MySQL, pleinement pris en charge par Laragon et accessible via la library Node.js mysql2. Ce choix est justifié par :

- la compatibilité avec le serveur Laragon déjà installé au sein de l'établissement ;
- la maîtrise des bases relationnelles pour modéliser les plannings d'affectations et créneaux ;
- la présence, dans MariaDB, de fonctionnalités clés pour faire appliquer les règles de gestion en base : vues (pour le calcul automatique des heures) et triggers (pour les contraintes métier avancées – interdiction de chevauchement de créneaux, contrôle du rattachement chef/service…).

CHAPITRE 5 : CONCEPTION DU PROJET

5.1. Présentation de la méthode de conception

La conception du projet a été réalisée avec la notation UML et une approche orientée objets. Le modèle du domaine est d'abord construit à partir du vocabulaire métier (service, employé, planning, affectation, créneau), puis les cas d'utilisation sont décrits, les diagrammes de séquence système précisent les échanges entre acteurs et le système, et la conception détaillée

traduit ces éléments en architecture logicielle (routes, contrôleurs, accès base) et en modèle de données relationnel.

5.2. Dictionnaire des données

Le tableau ci-dessous présente un extrait significatif du dictionnaire de données. L'intégralité des tables est décrite dans la partie base de données (Chapitre 6).

Tableau 5.1 — Extrait du dictionnaire de données

Entité / Attribut	**Type**	**Signification / Contraintes**
service.nom_service	VARCHAR(100) UNIQUE	Nom du service (Cuisine Snack, Salle Snack, Patisserie & Pizza…)
service.type_affichage	ENUM	Format de planning du service : PERIODES_FIXES, HORAIRE_LIBRE, SHIFT_POSTE
utilisateur.login / mot_de_passe	VARCHAR UNIQUE / VARCHAR(255)	Identifiant de connexion et mot de passe haché (bcrypt)
utilisateur.role	ENUM	ADMIN, RH ou CHEF_SERVICE
employe.matricule	VARCHAR(50) UNIQUE optionnel	Matricule ; les stagiaires n'en possèdent pas
employe.statut	ENUM	TITULAIRE ou STAGIAIRE
employe.reference_hebdo	DECIMAL(5,2) > 0 et ≤ 60	Référence hebdomadaire de comparaison (44 h ou 48 h)
planning.semaine / statut	DATE / ENUM	Semaine concernée ; BROUILLON, SOUMIS, VALIDE ou REJETE
affectation.statut_jour	ENUM	TRAVAIL, REPOS ou ABSENCE
creneau_travail.type_periode	ENUM	matin, apres-midi ou soir
creneau_travail.heure_debut / heure_fin	TIME	Bornes du créneau (le trigger interdit l'inversion)
validation_rh.decision / commentaire	ENUM / TEXT	VALIDE ou REJETE, accompagné d'un commenté motivé en cas de rejet
besoin_couverture.effectif_requis	INT	Effectif souhaité pour un jour et un créneau donnés

historique_modification.operation	VARCHAR	Nature de l'action tracée (création, affectation, soumission, validation…)

5.3. Règles de gestion

Tableau 5.2 — Règles de gestion

N°	Règle
RG1	Chaque employé est identifié par un seul champ nom ; le matricule est optionnel (campagnes de stagiaires) ; statuts TITULAIRE ou STAGIAIRE
RG2	Chaque jour d'un employé dans un planning a un statut parmi TRAVAIL, REPOS, ABSENCE ; une absence peut être précisée par un commentaire libre, sans sous-catégorie
RG3	Une journée de travail peut comporter jusqu'à 3 créneaux (matin, après-midi, soir), chacun avec heure d'entrée et heure de sortie
RG4	Chaque employé possède une référence hebdomadaire de 44 h ou 48 h, utilisée uniquement pour comparaison, sans blocage
RG5	Les totaux d'heures journalier et hebdomadaire ne sont jamais saisis : ils sont calculés automatiquement par des vues SQL (v_duree_creneau, v_duree_jour, v_total_hebdo_employe)
RG6	Cycle de vie du planning : BROUILLON → (soumission chef) → SOUMIS → (décision RH) → VALIDE ou REJETE ; SOUMIS et VALIDE sont non modifiables par le chef
RG7	Un dépassement de la référence hebdomadaire est signalé automatiquement, sans bloquer la saisie
RG8	Pour chaque jour et chaque créneau, un effectif requis peut être défini ; l'effectif réel affecté est comparé automatiquement pour détecter sous- ou sur-couverture
RG9	Les récupérations sont saisies uniquement par le chef de service ou la RH
RG10	Toute création ou modification significative d'un planning est enregistrée dans un historique horodaté, associée à l'utilisateur qui a réalisé l'action
RG11	Un planning est unique par service et par semaine
RG12	Un chef de service est rattaché à exactement un service (générée par colonne calculée et triggers trg_utilisateur_bi/bu)

5.4. Diagramme de cas d'utilisation

Le diagramme ci-dessous synthétise les cas d'utilisation identifiés. Les acteurs sont le chef de service et la RH, regroupés au sens large sous un acteur « utilisateur connecté » pour les cas communs (connexion, profil, messagerie). Les permissions sont ensuite affinées côté serveur par la matrice de permissions (Chapitre 7).

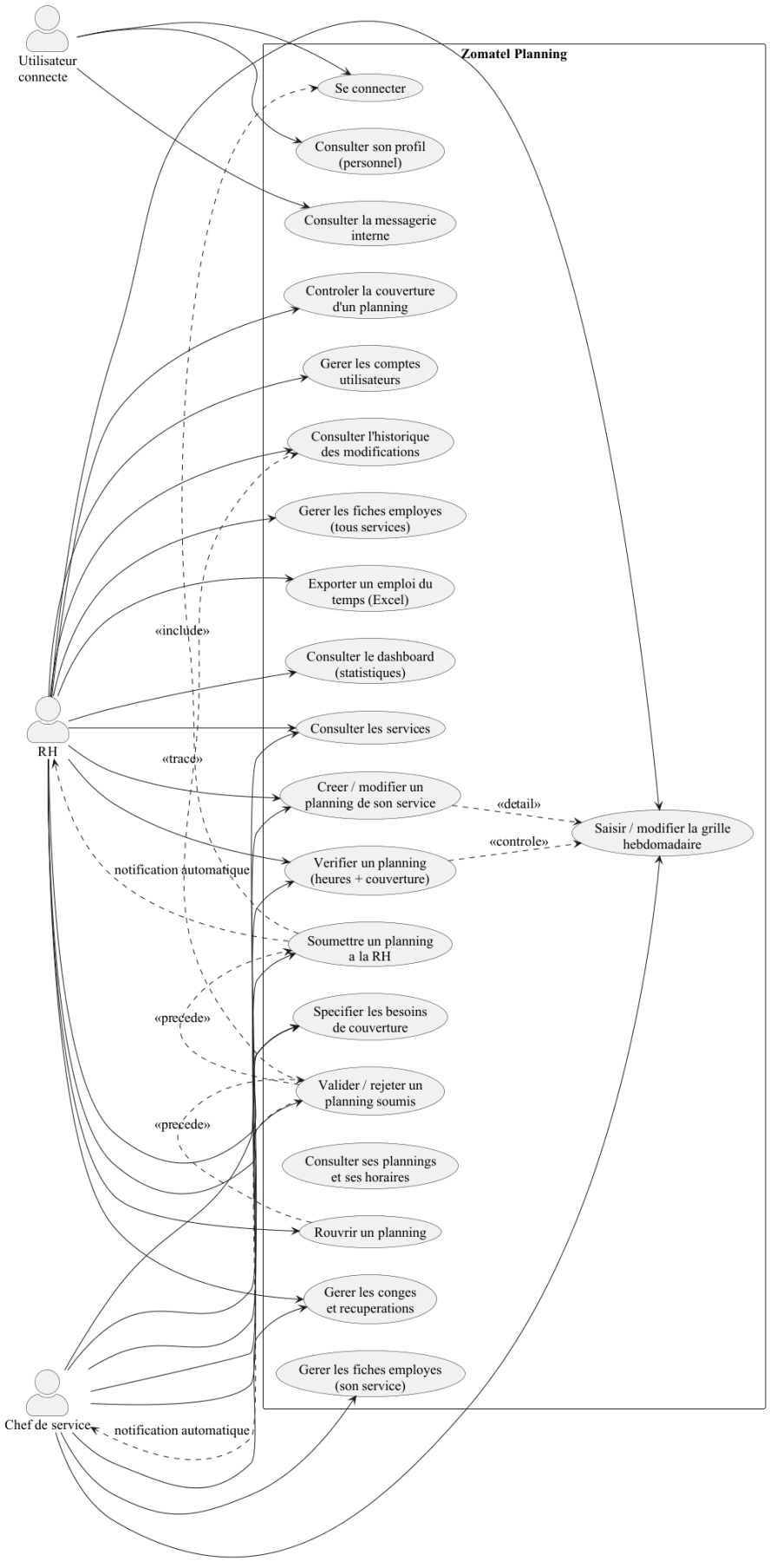

Zomatel Planning
Utilisateur connecte
RH
Chef de service
Se connecter
Consulter son profil (personnel)
Consulter la messagerie interne
Controler la couverture d'un planning
Gerer les comptes utilisateurs
Consulter l'historique des modifications
Gerer les fiches employes (tous services)
Exporter un emploi du temps (Excel)
Consulter le dashboard (statistiques)
Consulter les services
Creer / modifier un planning de son service
Verifier un planning (heures + couverture)
Soumettre un planning a la RH
Specifier les besoins de couverture
Valider / rejeter un planning soumis
Consulter ses plannings et ses horaires
Rouvrir un planning
Gerer les conges et recuperations
Gerer les fiches employes (son service)
Saisir / modifier la grille hebdomadaire
«include»
«trace»
«detail»
«controle»
notification automatique
«precede»
«precede»
notification automatique

Figure 4.2 — Diagramme de cas d'utilisation de l'application

5.5. Description textuelle des cas d'utilisation

Tableau 5.3 — Description textuelle du cas d'utilisation « Soumettre un planning à la RH »

Élément	**Description**
Cas d'utilisation	Soumettre un planning à la RH (cas du chef de service)
Acteur	Chef de service
Précondition	Le planning existe, statut BROUILLON ou REJETE, et appartient au service du chef
Scénario nominal	1) Le chef ouvre la grille hebdomadaire ; 2) Il clique « Soumettre à la RH » ; 3) Le système vérifie le statut et les contrôles automatiques ; 4) Le statut passe à SOUMIS ; 5) Une notification est envoyée à tous les comptes RH
Scénarios alternatifs	4a) Le planning contient des erreurs bloquantes (créneau invalide) : le système refuse la soumission et affiche le détail ; 5a) La RH rejette avec un motif : le planning repasse à REJETE et devient à nouveau modifiable
Postcondition	Le planning est au statut SOUMIS ; le chef ne peut plus le modifier

Tableau 5.4 — Description textuelle du cas d'utilisation « Valider ou rejeter un planning soumis »

Élément	**Description**
Cas d'utilisation	Valider ou rejeter un planning soumis (cas de la RH)
Acteur	RH
Précondition	Le planning est au statut SOUMIS
Scénario nominal	1) La RH ouvre la file de validations ; 2) Elle consulte le planning, les heures calculées et la couverture ; 3) Elle approuve ou rejette avec un commentaire ; 4) Le résultat est enregistré dans validation_rh ; le statut du planning passe à VALIDE ou REJETE ; 5) Une notification est adressée au chef créateur
Scénarios alternatifs	3a) Le planning Valve devient l'emploi du temps officiel et n'est plus modifiable ; 3b) Un planning rejeté peut être rouvert par la RH
Postcondition	La décision est enregistrée et horodatée ; l'historique est mis à jour

5.6. Diagrammes de séquence système

5.6.1. Soumission et validation d'un planning

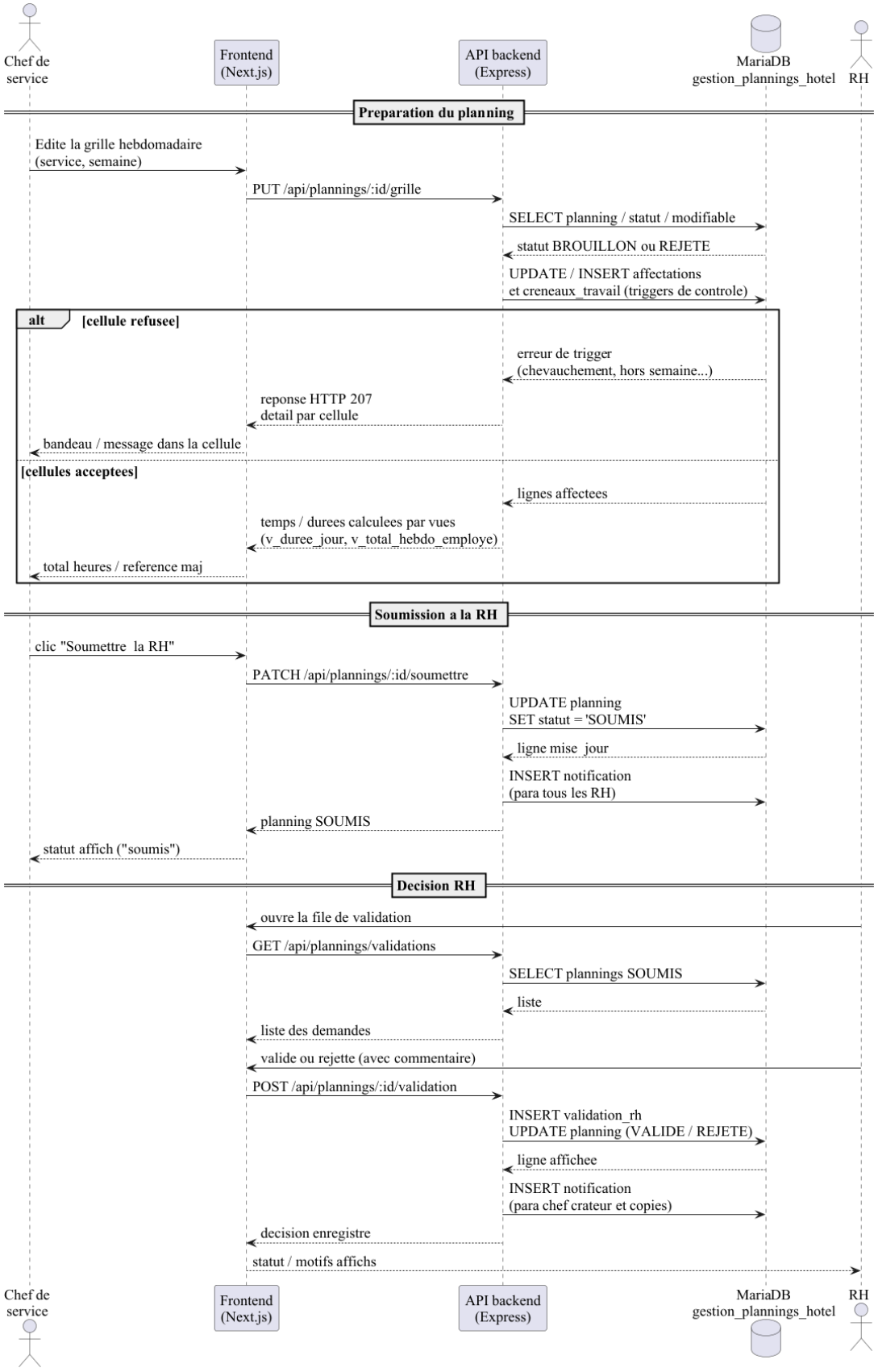


Figure 5.1 — Diagramme de séquence système : soumission et validation d'un planning

5.6.2. Authentification (login)

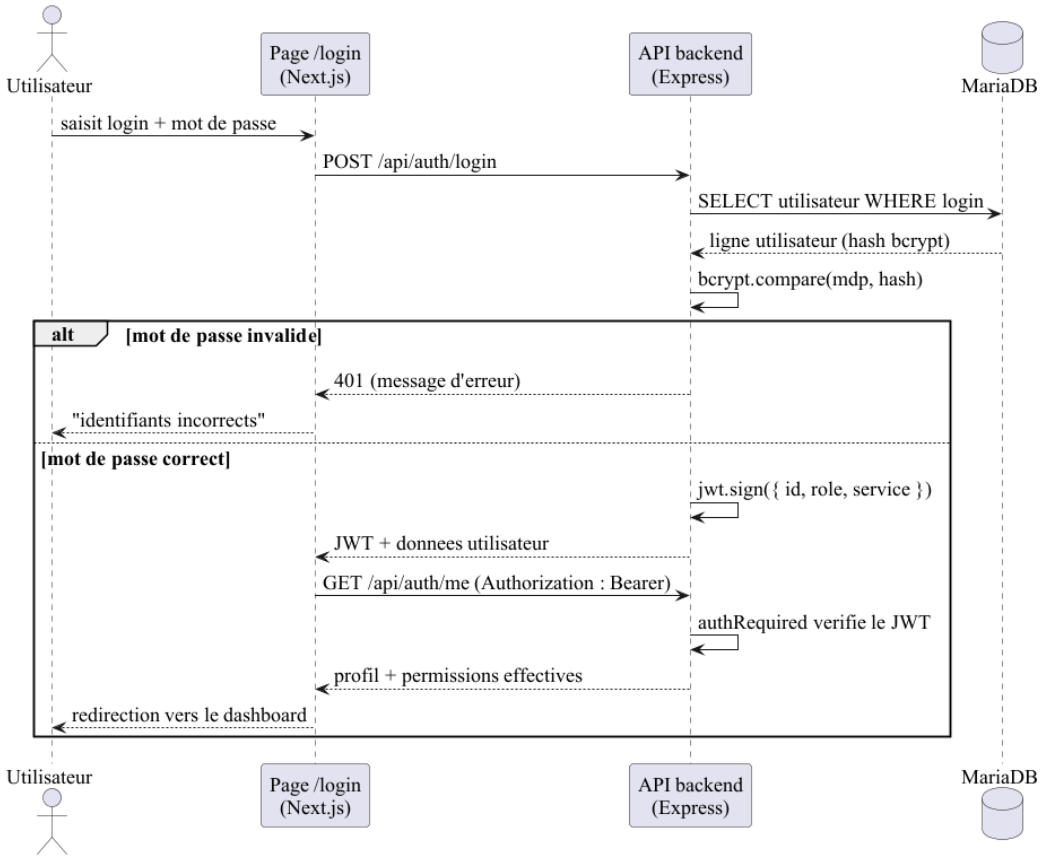


Figure 5.2 — Diagramme de séquence système : authentification

5.7. Modèle de domaine

Le modèle du domaine traduit le vocabulaire métier retenu. Le diagramme de classes détaillé au niveau de la base de données est présenté au chapitre 6.

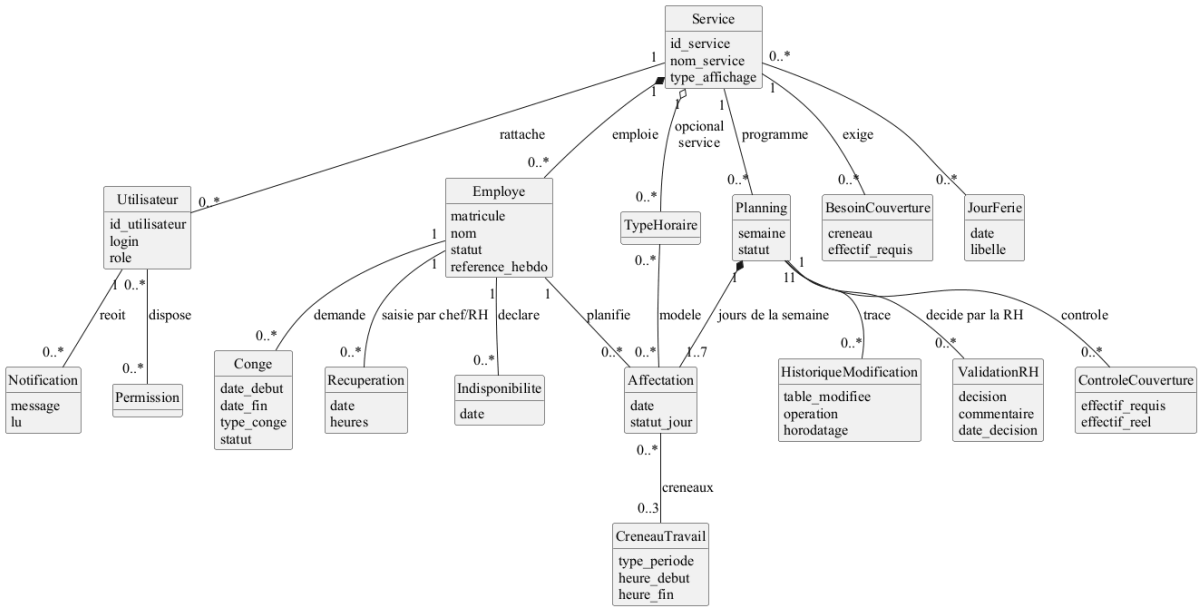


Figure 5.3 — Modèle de domaine

CHAPITRE 6 : CONCEPTION DÉTAILLÉE

6.1. Architecture de l'application

L'application est architecturée en trois tiers :

- un tier Client : l'interface web Next.js 14 (React 18), rendue par le navigateur, qui consomme l'API REST ;
- un tier Application : un serveur Node.js exécutant l'API Express, chargé de l'authentification (JWT), des permissions, des contrôleurs métier et des services (export Excel, e-mails) ;
- un tier Données : la base MariaDB gestion_plannings_hotel, qui porte le schéma relationnel, les vues de calcul des heures et les triggers de contrainte.

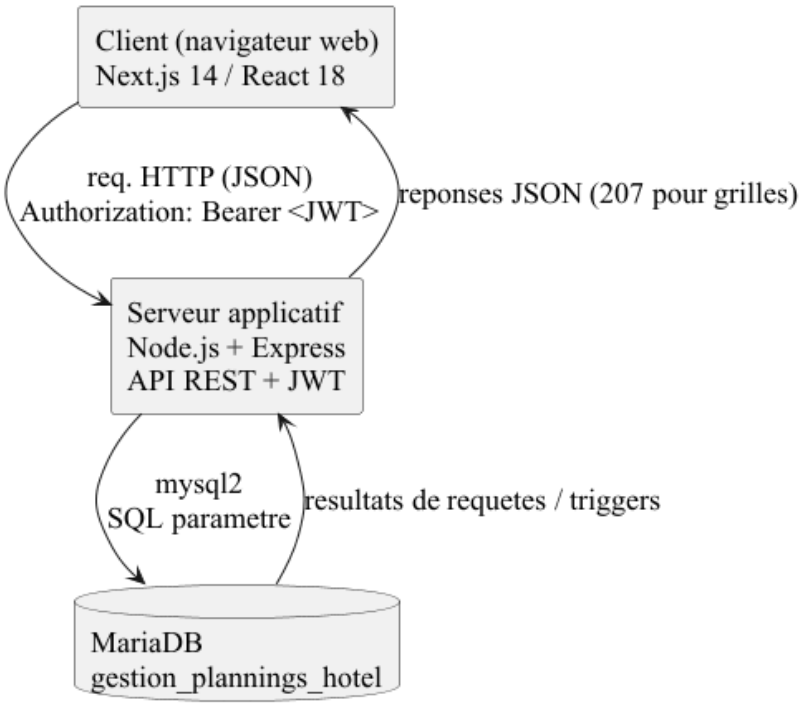


Figure 3.1 — Architecture trois tiers de l'application

Ce découpage répond à deux exigences : la séparation claire des responsabilités (interface / logique métier / données) et l'extensibilité de l'API, qui pourra servir plus tard une application mobile ou des intégrations tierces sans réécrire la logique métier.

6.1.1. Périmètre des rôles (matrice de permissions)

Conformément au cahier des charges, l'application ne comporte que deux rôles applicatifs : RH et CHEF_SERVICE ; l'employé n'a pas de compte (il est géré comme une fiche), et le rôle ADMIN n'est pas prévu par le système cible.

- CHEF_SERVICE : gestion des plannings de son service (grille hebdomadaire), fiches employés de son service, soumission à la RH, récupérations de son équipe ;
- RH : consultation de tous les services, contrôle et validation ou rejet des plannings soumis, gestion des comptes utilisateurs, fiches employés de tous services, consult de l'historique et export des emplois du temps.

Les permissions effectives sont vérifiées à chaque requête côté serveur (middleware requirePermission) après relecture du profil complet /api/auth/me, ce qui évite tout contournement par le client.

6.2. Diagramme de séquence de conception

Le diagramme de séquence système de la figure 5.1 est traduit au niveau de la conception par le chaînage concret suivant : le composant frontend (page /planning/gestion) appelle la route Express concernée ; la route applique d'abord le middleware authRequired (vérification du JWT), puis requirePermission ; le contrôleur consulte la base par le pool mysql2 et exécute les requêtes SQL ; la réponse est renvoyée en JSON, avec un statut HTTP 207 dans le cas d'une sauvegarde de grille dont certaines cellules sont refusées par les triggers — le détail cellule par cellule permet au frontend d'afficher l'erreur au bon endroit.

6.3. Diagramme de classes

Le diagramme de classes ci-dessous se concentre sur la couche applicative backend (contrôleurs, middlewares et services), le modèle de données relationnel étant détaillé à la section 6.5. Les contrôleurs regroupent les opérations décrites dans les cas d'utilisation, les middlewares sécurisent l'accès, et les services embarquent la logique de spécialité (export Excel, e-mails).

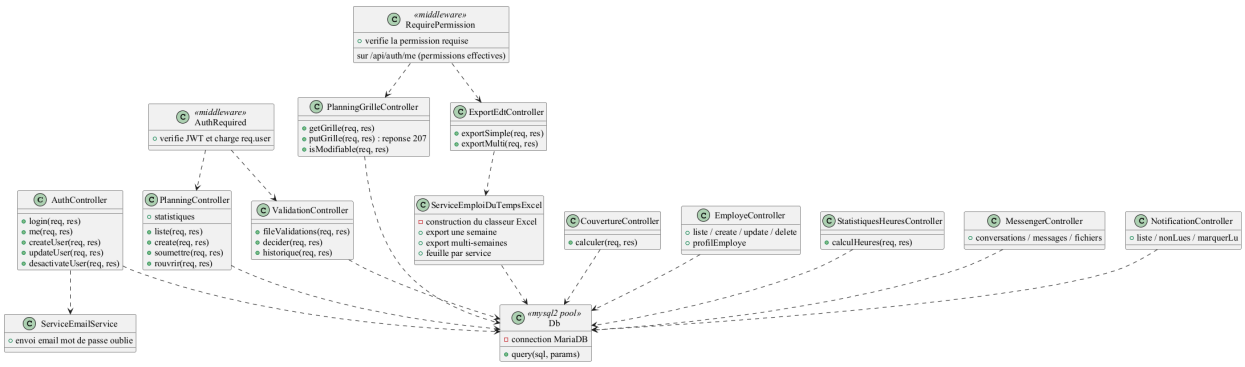


Figure 6.4 — Diagramme de classes backend

6.4. Diagramme de paquetages

Le découpage en paquetages reflète l'organisation réelle des répertoires du projet : côté frontend, le dossier app (App Router) regroupe les pages, le dossier lib les utilitaires transverses (client API, authentification, permissions, données de planning, socket) ; côté backend, les paquetages routes, controllers, middleware, services et socket ; côté données, la base MariaDB avec ses tables, vues et triggers.

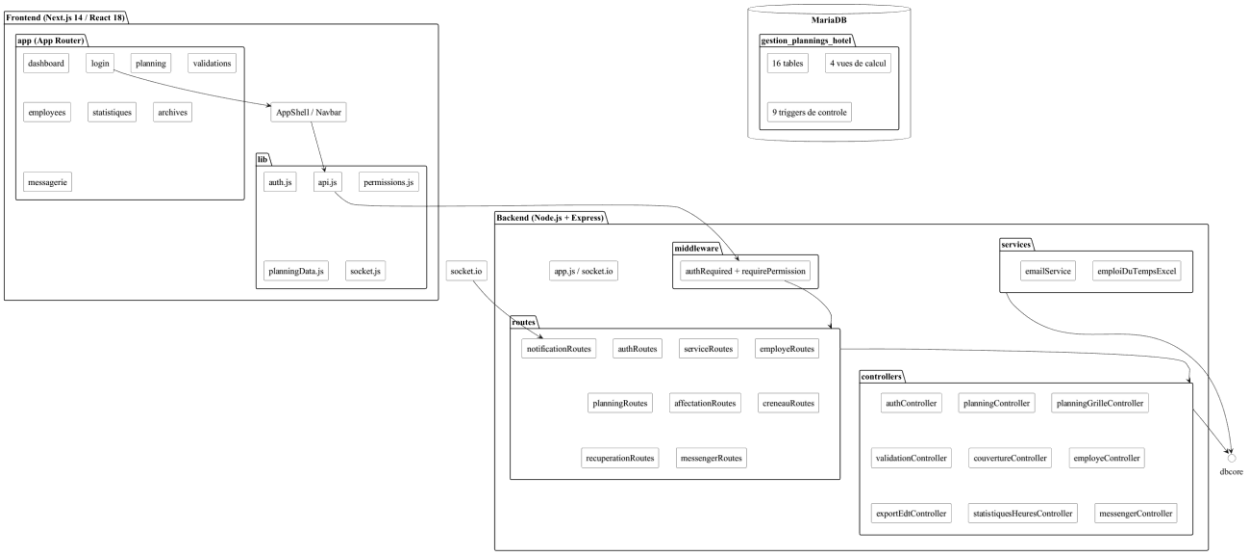


Figure 6.1 — Diagramme de paquetages de l'application

6.5. Diagramme de déploiement

Le déploiement cible est volontairement léger, adapté à la taille de l'établissement et à son infrastructure existante : les deux serveurs (Next.js et l'API Express) et la base MariaDB tournent sur un serveur local Windows (Laragon). Les postes du personnel de gestion accèdent à l'application depuis leur navigateur.

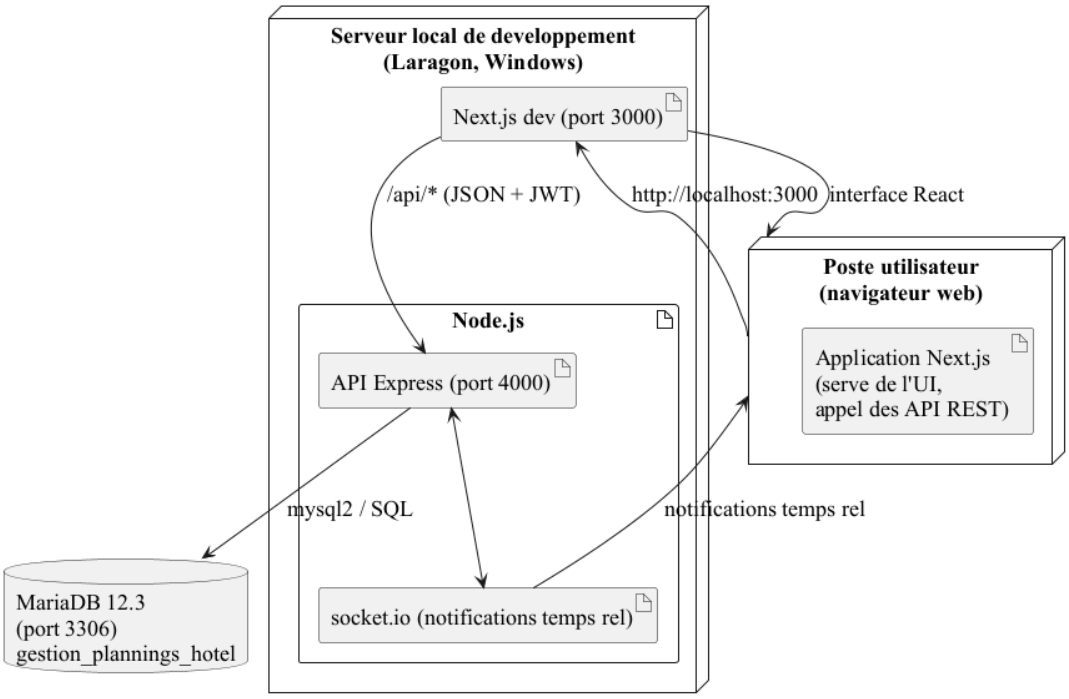


Figure 6.2 — Diagramme de déploiement

6.6. Modèle conceptuel des données

La base gestion_plannings_hotel comporte 16 tables, 4 vues et 9 triggers. Le modèle conceptuel ci-dessous décrit les entités principales et leurs cardinalités. Le tableau 6.1 précise ensuite le rôle de chacune de ces structures.

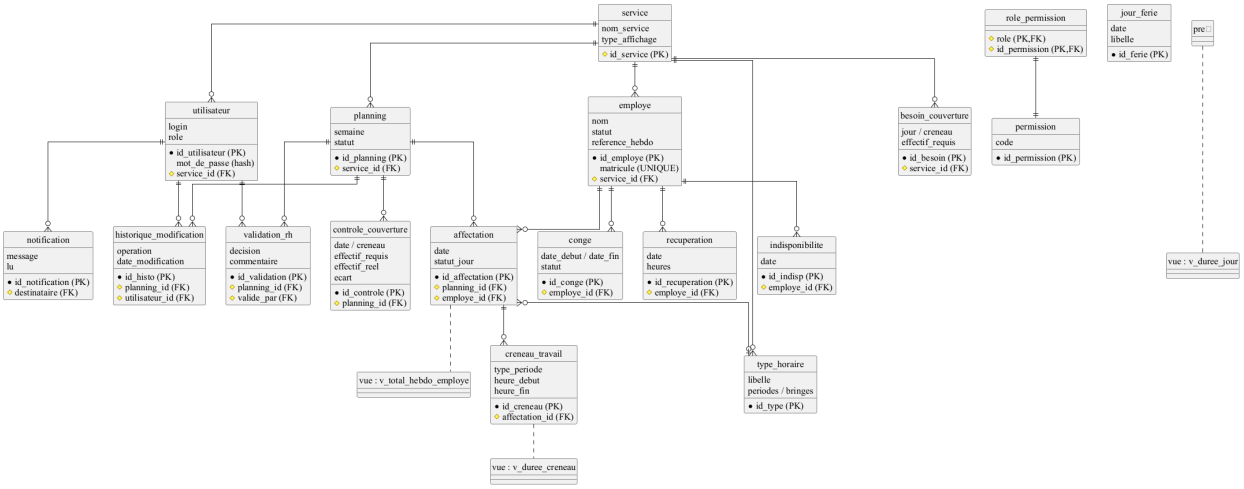


Figure 6.3 — Modèle conceptuel des données (MCD)

Tableau 6.1 — Synthèse de la base de données gestion_plannings_hotel

Table	**Rôle**
service	Liste des services, avec le format de planning choisi (PERIODES_FIXES, HORAIRE_LIBRE, SHIFT_POSTE)
utilisateur	Comptes de connexion ; rôle ; rattachement du chef à un service unique (colonne générée + triggers)
permission / role_permission	Matrice de permissions par rôle, revue à chaque requête via /api/auth/me
employe	Fiche du personnel : matricule optionnel, nom, statut TITULAIRE/STAGIAIRE, service, référence hebdomadaire
type_horaire	Modèles d'horaires réutilisables par service
planning	Un planning = un service + une semaine + un statut (BROUILLON, SOUMIS, VALIDE, REJETE)
affectation	Statut d'un employé pour un jour du planning (TRAVAIL, REPOS, ABSENCE)
creneau_travail	Jusqu'à 3 créneaux par affectation : matin, après-midi, soir, avec heure

	d'entrée et heure de sortie
conge	Congés et indisponibilités communiqués par la RH, avec contrôle de statut (trigger)
recuperation	Récupérations saisies par chef ou RH (trigger de contrôle du rôle)
indisponibilite	Jours d'indisponibilité ponctuelle d'un employé
validation_rh	Décision RH sur un planning soumis (VALIDE/REJETE + commentaire + date)
controle_couverture	Résultat du contrôle effectif requis / effectif réel, par jour et par créneau
besoin_couverture	Effectifs requis définis par service, par jour et créneau
historique_modification	Journal horodaté des actions réalisées sur un planning, avec l'utilisateur à l'origine
jour_ferie	Liste des jours fériés
notification	Notifications applicatives (soumission, décision), avec état lu / non lu
Vues	v_duree_creneau, v_duree_jour, v_total_hebdo_employe (calculs d'heures), v_emploi_du_temps

Tableau 6.2 — Principales règles métier appliquées par triggers MariaDB

Règle appliquée en base	**Trigger**
Refus des conflits d'horaires (chevauchement de créneaux)	trg_creneau_bi / trg_creneau_bu
Interdiction d'un créneau sur un jour de repos / absence / congé	trg_creneau_bi / trg_creneau_bu
Affectation bornée à la semaine du planning	trg_affectation_bi
Validation d'un planning réservée aux comptes RH	trg_validation_rh_bi
Congé validé ou refusé uniquement par un RH, avec traçabilité	trg_conge_bi / trg_conge_bu
Récupération saisie par RH, ADMIN ou CHEF_SERVICE	trg_recuperation_bi
CHEF_SERVICE toujours rattaché à un service	trg_utilisateur_bi / trg_utilisateur_bu
Maximum 3 créneaux (matin / après-midi / soir) par jour	trg_creneau_bi

PARTIE 3 : MISE EN ŒUVRE ET RÉSULTATS

CHAPITRE 7 : MISE EN ŒUVRE

7.1. Architecture du système

L'implémentation reprend exactement l'architecture de conception : deux processus (frontend Next.js sur le port 3000, API Express sur le port 4000) et une base MariaDB. Les échanges internes se font en JSON sur HTTP ; les notifications temps réel (postes connectés) passent par socket.io côté backend et socket.io-client côté frontend. Le stockage des photos de profil et des pièces jointes de la messagerie se fait dans le dossier uploads du backend, servi par multer.

7.1.1. Organisation du backend

Le backend est organisé par responsabilités :

- src/routes : définition des points d'entrée REST par domaine (auth, services, employés, plannings, affectations, créneaux, types d'horaires, récupérations, messagerie, notifications) ;
- src/controllers : logique de traitement par domaine (soumission, validation, grille, couverture, export EDT, statistiques d'heures…) ;
- src/middleware : authRequired (JWT) et requirePermission (permissions par rôle) ;
- src/services : services encapsulés (emploiDuTempsExcel pour l'export, emailService pour l'oubli de mot de passe) ;
- src/socket : notifications temps réel ;
- src/config : configuration centralisée (connexion base, secret JWT).

7.1.2. Organisation du frontend

Le frontend suit l'App Router de Next.js : chaque page (dashboard, planning, validations, employees, statistiques, archives, messagerie, paramètres, utilisateurs, profile) est un dossier de src/app ; les composants d'interface sont regroupés dans src/components (grilles du planning, éditeurs par format, navbar, notifications, graphiques du dashboard…) ; les utilitaires transverses sont dans src/lib (client API avec jeton JWT, gestion d'authentification, permissions, i18n).

7.1.3. Authentification et permissions

- Toutes les routes exigent un jeton JWT valide (middleware authRequired), y compris /api/services ;
- Le jeton encode l'identité, le rôle et le service de l'utilisateur ; il est vérifié à chaque appel de l'API ;
- Les mots de passe sont stockés hachés (bcrypt), jamais en clair ;
- Les permissions effectives sont relues à partir de /api/auth/me, ce qui permet à la navigation du frontend de n'afficher que les actions autorisées, tout en gardant la décision finale côté serveur (requirePermission) ;
- Le chef de service reste limité à son service : un compte sans la permission planning.view_all ne voit que le service qui lui est associé.

7.2. Configuration des outils de conception

- PlantUML : diagrams en textual-généré, compilés en PNG pour le rapport ;
- Visual Studio Code : extensions JavaScript/React et SQL ;
- Git : gestion de versions, dépôt local du projet ;
- HeidiSQL (fourni avec Laragon) : exploration visuelle de la base MariaDB pendant le développement.

7.3. Génération de la base de données

La base gestion_plannings_hotel est créée par exécution de deux scripts SQL fournis dans backend/database :

```
mysql -u root -p < backend/database/gestion_plannings_hotel_schema.sql
```

```
mysql -u root -p < backend/database/gestion_plannings_hotel_seed.sql
```

Le script de schéma crée la base, les 16 tables, les 4 vues de calcul et les 9 triggers de contrôle ; le script de jeu de données insère les services, les comptes de démonstration, les employés et quelques plannings d'exemple, dont la structure reprend celle du fichier Excel de référence de l'établissement (ex. : ANYAH, entrée 06:00, sortie 12:00, etc.). Des scripts de migration complémentaires activent des fonctionnalités sur une base existante (09_service_model_types.sql pour les formats SHIFT_POSTE, 10_one_chef_per_service.sql pour l'unicité du chef par service, migrate-10-planning-export.js pour la permission d'export).

7.4. Configuration de l'environnement de développement

1) dans backend : npm install, puis configuration du fichier .env (connexion MariaDB, secret JWT), puis npm run dev (nodemon) ;

2) dans frontend : npm install, configuration de l'URL de l'API, puis npm run dev (next dev) ;

3) base : création par les scripts SQL ci-dessus, accessible via MariaDB 12.3 de Laragon ;

4) accès : http://localhost:3000 pour l'application, http://localhost:4000 pour l'API.

Les comptes de démonstration fournis couvrent chaque rôle (admin123, rh123, chef123, employe123 selon README) et permettent de tester le comportement de chaque profil.

7.5. Extraits de code des fonctionnalités principales

7.5.1. Authentification (login) et génération du JWT

Extrait authentifié de la méthode de connexion (backend/src/controllers/authController.js), abrégé pour le rapport :

```
// Post /api/auth/login
const user = await db.query(
  'SELECT * FROM utilisateur WHERE login = ? AND actif = 1 LIMIT 1', [login]);
if (!user.length || !(await bcrypt.compare(motDePasse, user[0].mot_de_passe)))
  return res.status(401).json({ message: 'Identifiants invalides' });
const token = jwt.sign(
  { id: user[0].id_utilisateur, role: user[0].role, service:
user[0].service_id },
  process.env.JWT_SECRET, { expiresIn: '8h' });
res.json({ token, utilisateur: sanitize(user[0]) });
```

7.5.2. Sauvegarde de la grille du planning avec réponse 207

La sauvegarde de grille (planningGrilleController.js) applique ligne par ligne les modifications, et remonte cellule par cellule les refus issus des triggers :

```
// PUT /api/plannings/:id/grille
const results = []; let refused = 0;
for (const cell of cells) {
  try { await applyCellule(cell); results.push({ cell: cell.ref, ok: true });
}
  catch (e) {
    refused++;
    results.push({ cell: cell.ref, ok: false, error: e.message });
  }
}
if (refused === cells.length) return res.json({ partial: false, results });
return res.status(207).json({ partial: true, results });
```

7.5.3. Contrôle en base : trigger d'interdiction de chevauchement

Extrait de trg_creneau_bi (backend/database/gestion_plannings_hotel_schema.sql), qui refuse l'enregistrement d'un créneau chevauchant un créneau existant du même employé :

```
CREATE TRIGGER trg_creneau_bi BEFORE INSERT ON creneau_travail
FOR EACH ROW BEGIN
  DECLARE n INT;
  SELECT COUNT(*) INTO n FROM creneau_travail c JOIN affectation a
    ON c.affectation_id = a.id_affectation
   WHERE a.employe_id = (SELECT employe_id FROM affectation
                         WHERE id_affectation = NEW.affectation_id)
     AND a.date = (SELECT date FROM affectation
                   WHERE id_affectation = NEW.affectation_id)
     AND c.id_creneau <> NEW.id_creneau
     AND NEW.heure_debut < c.heure_fin
     AND c.heure_debut < NEW.heure_fin;
  IF n > 0 THEN
    SIGNAL SQLSTATE '45000'
      SET MESSAGE_TEXT = 'Chevauchement de créneaux interdit';
  END IF;
END
```

7.5.4. Calcul automatique des heures (vues SQL)

Extrait de v_duree_creneau puis de v_total_hebdo_employe :

```
CREATE OR REPLACE VIEW v_duree_creneau AS
SELECT id_creneau,
       TIMESTAMPDIFF(MINUTE, heure_debut, heure_fin) / 60.0 AS duree
  FROM creneau_travail;

CREATE OR REPLACE VIEW v_duree_jour AS
SELECT a.id_affectation, a.planning_id, a.employe_id, a.date,
       COALESCE(SUM(dc.duree), 0) AS duree_jour
  FROM affectation a
  LEFT JOIN creneau_travail c ON c.affectation_id = a.id_affectation
  LEFT JOIN v_duree_creneau dc ON dc.id_creneau = c.id_creneau
 GROUP BY a.id_affectation;

CREATE OR REPLACE VIEW v_total_hebdo_employe AS
SELECT planning_id, employe_id, SUM(duree_jour) AS total_hebdo
  FROM v_duree_jour GROUP BY planning_id, employe_id;
```

7.5.5. Export Excel de l'emploi du temps (service)

L'export (services/emploiDuTempsExcel.js) construit le classeur Excel réservé RH + permission planning.export_edt : une feuille par service, en respectant la structure du fichier historique de l'établissement (créneaux matin / après-midi / soir, durée calculée).

Côté frontend, la grille de saisie (components/PlanningEditorShiftPoste.js, PlanningEditorHoraireLibre.js, AffectationsHoraireLibre.js) utilise une structure simple : chaque cellule se modifie par un clic, les totaux hebdomadaires se recalculent à partir de la réponse du serveur (v_total_hebdo_employe), et la comparaison à la référence se traduit par une alerte visuelle de dépassement.

CHAPITRE 8 : PRÉSENTATION DE L'APPLICATION

L'interface se parcourt selon une ligne de démonstration : connexion → dashboard → gestion du planning → soumission → validation RH → consultation / export. Les figures ci-dessous illustrent cette logique. Les captures d'écran actualisées sont à insérer à l'emplacement indiqué (cf. ANNEXE pour la démarche).

8.1. Page de connexion

8.2. Tableau de bord (dashboard)

8.3. Gestion du planning (grille hebdomadaire)

8.4. Statuts et soumission

8.5. Validation RH

8.6. Gestion des employés

8.7. Messagerie interne et notifications

Capture d'écran de l'interface — À COMPLÉTER : insérer ici la capture correspondante (répertoire captures/ ou à générer en phase de finition).

CHAPITRE 9 : ÉVALUATION ET SUGGESTIONS

9.1. Évaluation de l'application (comparaison avec des solutions existantes)

Tableau 9.1 — Comparaison de l'application avec des solutions existantes

Critère	**Plannings Excel actuels**	**Application Zomatel Planning**
Calcul des heures	Formules de tableur, erreurs fréquentes	Automatique par vues SQL, jamais saisi
Circuit de validation	Aucun	Formalisé (BROUILLON → SOUMIS → VALIDE/REJETE)
Traçabilité des modifications	Aucune	Historique horodaté de toutes les actions
Conflits d'horaires	Détectés visuellement (ou pas)	Refus automatique en base (trigger)

Couverture de service	Vérification manuelle	Comparaison automatique effectif requis / réel
Notifications	Par WhatsApp ou déplacement	Ciblées et temps réel (socket.io)
Export à la paie	Consolidation manuelle	Export Excel structuré par service et par semaine
Dépendance réseau/serveur	Aucune (fichiers locaux)	Nécessite un serveur applicatif et la base installée

Cette comparaison montre que l'application répond point par point aux limites relevées dans l'analyse de l'existant. Elle ne remplace pas la rigueur des responsables : elle rend les contrôles automatiques et l'information partagée.

9.2. Contributions académiques et professionnelles

- Académiques : mise en pratique complète d'un cycle de développement (analyse, conception UML, développement client-serveur, tests), apprentissage des vues et triggers MariaDB, exploitation d'une architecture REST sécurisée par JWT ;
- Professionnelles : découverte du fonctionnement concret d'un établissement hôtelier (services, contraintes d'horaires, paie), capacité d'adaptation aux règles de gestion réelles plutôt qu'à un modèle théorique, dialogue avec des utilisateurs non informaticiens.

9.3. Limitations de l'étude et pistes de recherche futures

9.3.1. Limites

- Pas de rôle ADMIN prévu dans le système cible, conformément au cahier des charges ; le rôle reste néanmoins présent dans le schéma de la base (cf. compte de démonstration) ;
- Pas de sous-catégorie d'absence (pas de distinction congé / maladie / congé sans solde) ;
- Pas de contrôle obligatoire d'un volume horaire journalier fixe (pas de règle des 8 h/jour) ;
- Les employés ne disposent pas de compte de connexion et ne peuvent pas demander eux-mêmes une modification ou une récupération ;
- Le déploiement cible reste local (Laragon) ; la mise en production sur un serveur de l'établissement et les sauvegardes restent à organiser ;
- Les captures d'écran finales et les dates exactes du stage restent à compléter dans le présent rapport.

9.3.2. Perspectives

- Portail employé : consultation de son planning et demande de récupération, avec notification au chef de service ;
- Application mobile de consultation ;
- Planification pluri-hebdomadaire (vue mensuelle, prévision des heures sur plusieurs semaines) ;
- Statistiques avancées (heures par service, taux de couverture, tendances) et exports élargis ;
- Intégration avec la paie ;
- Internationalisation complète de l'interface (bases déjà en places via /lib/i18n.js).

CONCLUSION GÉNÉRALE ET PERSPECTIVES

Ce stage avait pour objet la digitalisation de la gestion des plannings du personnel de l'établissement ZOMATEL. Partant d'un processus manuel constitué de fichiers Excel indépendants par service, il a abouti à une application web structurée en trois tiers (Next.js 14, API Node.js/Express, base MariaDB) qui couvre le cycle complet : saisie en grille façon tableur, calcul automatique des heures par des vues SQL, contrôles de cohérence en base par des triggers, circuit de validation formel entre chefs de service et RH, notifications automatiques et temps réel, traçabilité horodatée des actions et export Excel des emplois du temps.

La solution retenue correspond aux règles de gestion observées sur le terrain et formalisées dans le cahier des charges : jalonnement des statuts (BROUILLON, SOUMIS, VALIDE, REJETE), identité simplifiée des employés (nom, matricule optionnel, statut titulaire ou stagiaire), référence hebdomadaire de 44 ou 48 heures avec alerte de dépassement sans blocage, gestion des repos, absences, congés et récupérations, contrôle de couverture par créneau.

Les points forts de l'application développée sont : l'automatisation complète des calculs (aucun total saisi par l'utilisateur), la fiabilité au plus près des données (triggers MariaDB), la traçabilité de bout en bout, la délimitation claire des responsabilités (un chef = un service), l'ergonomie de saisie comparable à un tableur, et l'extensibilité (nouveaux services et formats de planning sans modification du modèle de données).

Les difficultés rencontrées lors de la mise en œuvre tiennent essentiellement à trois points : la modélisation des règles de gestion réelles (qui devaient correspondre exactement à la pratique de l'établissement, non au contraire d'un modèle théorique), la mise en cohérence des contrôles entre client, serveur et base (la réponse HTTP 207 refusée cellule par cellule a demandé plusieurs itérations), et la contrainte matérielle de travailler sur un environnement local Windows (Laragon) tout en respectant les contraintes du parc de l'établissement.

En perspectives, la poursuite de ce travail pourrait porter sur un portail employé (consultation de son planning, demande de récupération), une application mobile, une planification pluri-hebdomadaire, des statistiques avancées et une intégration avec la paie. Plus globalement, ce stage confirme l'intérêt, pour une petite structure, d'une digitalisation ciblée : une application simple, adaptée aux usages réels, et des règles rendues automatiques valent mieux qu'un outil généraliste configuré à moitié.

BIBLIOGRAPHIE

OMG, 2017. OMG Unified Modeling Language (OMG UML), version 2.5.1 [en ligne]. Object Management Group. Disponible sur : <https://www.omg.org/spec/UML/2.5.1> (consulté le 08/10/2026).

Node.js Foundation, 2026. Node.js v22 documentation [en ligne]. Disponible sur : <https://nodejs.org/docs/latest-v22.x/api/> (consulté le 05/10/2026).

Express, 2026. Express 4.x API Reference [en ligne]. OpenJS Foundation. Disponible sur : <https://expressjs.com/en/4x/api.html> (consulté le 05/10/2026).

Next.js, 2026. Next.js 14 documentation [en ligne]. Vercel. Disponible sur : <https://nextjs.org/docs/14> (consulté le 05/10/2026).

React, 2026. React 18 documentation [en ligne]. Meta Platforms. Disponible sur : <https://react.dev/> (consulté le 05/10/2026).

MariaDB Corporation, 2026. MariaDB Server documentation [en ligne]. Disponible sur : <https://mariadb.com/kb/en/documentation/> (consulté le 05/10/2026).

Auth0 Inc., 2024. JSON Web Token (JWT) — RFC 7519 [en ligne]. IETF. Disponible sur : <https://datatracker.ietf.org/doc/html/rfc7519> (consulté le 06/10/2026).

Provos N. & Mazières D., 1999. A future-adaptable password scheme. In Proceedings of the 1999 USENIX Annual Technical Conference (FREENIX Track). Monterey : USENIX Association, pp. 81-91.

Fielding R., 2000. Architectural styles and the design of network-based software architectures. Th. doct. : Inf. : Irvine : University of California, Irvine, 180 p.

N.B. : les dates de consultation sont à actualiser par l'étudiant au moment de la soutenance (À COMPLÉTER). Les références suivent le modèle demandé par le canevas (Nom, année. Titre en italique. [en ligne]. Disponible sur : <URL> (consulté le ...)).

WEBOGRAPHIE

Wikipedia, 2026. Planning des ressources humaines [en ligne]. Disponible sur : <https://fr.wikipedia.org/wiki/Planning_des_ressources_humaines> (consulté le 06/10/2026).

OpenJS Foundation, 2026. socket.io documentation [en ligne]. Disponible sur : <https://socket.io/docs/v4/> (consulté le 06/10/2026).

Vercel, 2026. Next.js App Router documentation [en ligne]. Disponible sur : <https://nextjs.org/docs/app> (consulté le 06/10/2026).

MySQL / MariaDB, 2026. CREATE TRIGGER statement [en ligne]. MariaDB Knowledge Base. Disponible sur : <https://mariadb.com/kb/en/create-trigger/> (consulté le 06/10/2026).

Laragon, 2026. Laragon documentation [en ligne]. Disponible sur : <https://laragon.org/docs/> (consulté le 06/10/2026).

ANNEXES

Les annexes suivantes sont annoncées dans le corps du texte :

- Annexe A — Structure complète de la base de données gestion_plannings_hotel (script SQL de schéma).
- Annexe B — Jeu de données de démonstration (extrait du script de seed).
- Annexe C — Extrait du fichier Excel de référence de l'établissement (semaine 37 de 2026, feuille Cuisine Snack).

(voir Annexe A ci-dessous), (voir Annexe B ci-dessous) ; le contenu est présenté dans les pages suivantes, paginées en continuation du rapport.

ANNEXE A — STRUCTURE DE LA BASE DE DONNÉES

Extrait du script backend/database/gestion_plannings_hotel_schema.sql (tables principales) :

```
CREATE TABLE IF NOT EXISTS service (
  id_service INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  nom_service VARCHAR(100) NOT NULL,
  type_affichage ENUM('PERIODES_FIXES','HORAIRE_LIBRE','SHIFT_POSTE')
    NOT NULL DEFAULT 'PERIODES_FIXES',
  description VARCHAR(255) DEFAULT NULL,
  date_creation DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  UNIQUE KEY uq_service_nom (nom_service)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

CREATE TABLE IF NOT EXISTS utilisateur (
  id_utilisateur INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  nom_utilisateur VARCHAR(100) NOT NULL,
  prenom VARCHAR(100) DEFAULT NULL,
  login VARCHAR(100) NOT NULL,
  mot_de_passe VARCHAR(255) NOT NULL, -- hash bcrypt
  role ENUM('ADMIN','RH','CHEF_SERVICE') NOT NULL,
  service_id INT UNSIGNED DEFAULT NULL,
  chef_service_unique_id INT UNSIGNED GENERATED ALWAYS AS
    (CASE WHEN role = 'CHEF_SERVICE' THEN service_id ELSE NULL END) PERSISTENT,
  actif TINYINT(1) NOT NULL DEFAULT 1,
  UNIQUE KEY uq_utilisateur_login (login),
  UNIQUE KEY uq_utilisateur_chef_service (chef_service_unique_id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

CREATE TABLE IF NOT EXISTS employe (
  id_employe INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  matricule VARCHAR(50) DEFAULT NULL,
  nom VARCHAR(150) NOT NULL,
  statut ENUM('TITULAIRE','STAGIAIRE') NOT NULL DEFAULT 'TITULAIRE',
  fonction VARCHAR(100) DEFAULT NULL,
  service_id INT UNSIGNED NOT NULL,
  reference_hebdo DECIMAL(5,2) NOT NULL DEFAULT 44.00,
  date_entree DATE DEFAULT NULL,
  actif TINYINT(1) NOT NULL DEFAULT 1,
  CONSTRAINT chk_employe_reference CHECK (reference_hebdo > 0 AND reference_hebdo <= 60)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

ANNEXE B — JEU DE DONNÉES DE DÉMONSTRATION (EXTRAIT)

Les comptes de démonstration fournis avec la base (script de seed) couvrent chaque rôle :

Tableau B.1 — Comptes de démonstration

Login	**Mot de passe**	**Rôle**
admin	admin123	ADMIN
rh	rh123	RH
chef	chef123	CHEF_SERVICE (Réception)
employe	employe123	EMPLOYE (lié à RAMORA)

Jeu d'employés de démonstration (extrait, repris de la pratique de l'établissement) : matricules P477 ANYAH (06:00-12:00), P522 AVOTRA (10:00-14:00 et 18:00-22:00), P543 AUGUSTIN (14:00-18:00 et 18:00-22:00), etc.

ANNEXE C — FICHIER EXCEL DE RÉFÉRENCE (EXTRAIT)

Structure du fichier « EMPLOI DU TEMPS ZOMATEL WEEK 37_2026.xlsx », feuille « CUISINE SNACK », dont l'application reprend le modèle (format PERIODES_FIXES) est la suivante, pour chaque employé :

Tableau C.1 — Structure du fichier de référence (feuille Cuisine Snack)

Matricule	**Nom**	**matin entrée**	**matin sortie**	**durée**	**après-midi entrée**	**après-midi sortie**	**durée**	**soir entrée**	**soir sortie**
P477	ANYAH	06:00	12:00	6:00	12:00	14:00	2:00		
P522	AVOTRA	10:00	14:00	4:00			0:00	18:00	22:00
P543	AUGUSTIN			0:00	14:00	18:00	4:00	18:00	22:00
P404	EDHINO	06:00	12:00	6:00	12:00	14:00	2:00		
P472	HERMANN	06:00	12:00	6:00	12:00	14:00	2:00		

RÉSUMÉ + MOTS-CLÉS (COUVERTURE DE DOS)

Conformément au canevas, les versions française et anglaise du résumé et des mots-clés figurent également sur la couverture de dos du rapport imprimé. Elles reprennent, sans en modifier le contenu, le Résumé et l'Abstract figurant en pages préliminaires de ce rapport.

MOTS-CLÉS : planning du personnel ; hôtellerie ; digitalisation ; application web ; Next.js ; Node.js ; MariaDB ; validation RH ; heures de travail.

KEYWORDS: staff scheduling; hotel industry; digitization; web application; Next.js; Node.js; MariaDB; HR validation; working hours.